DATA ANALYTICS LAB PROGRAMS [SEP SYLLABUS]

EXPERIMENT NO. 1

Title

Probability Distribution of Categorical Variables Using Python

Aim

To calculate the probability distribution of categorical variables such as **Gender, Batting Style, Bowling Style, Player Position, and Country** from the given dataset using Python and the Pandas library.

Problem Statement

From the given dataset **players_info.csv**, perform the following tasks:

a) Calculate the probability distribution of genders among the players.

b) Calculate the probability distribution of batting styles.

c) Calculate the probability distribution of bowling styles.

d) Calculate the probability distribution of player positions.

e) Calculate the probability distribution of countries among the players.

Learning Objectives

After completing this experiment, the student will be able to:

- Understand the concept of probability distribution.
- Read a CSV dataset using the Pandas library.
- Calculate the frequency of categorical variables.
- Compute probability values using mathematical formulas.
- Display the results in tabular format using Python.
- Verify that the sum of all probabilities equals 1.

Theory

Data Analytics involves collecting, organizing, analysing, and interpreting data to obtain useful information. One of the fundamental statistical concepts used in data analysis is the **probability distribution**.

A probability distribution shows how the total observations in a dataset are distributed among different categories. It indicates the likelihood of occurrence of each category.

For categorical data, the probability of a category is calculated using the following formula:

$$P(E) = \frac{\text{Frequency of the Category}}{\text{Total Number of Records}}$$

Where:

- **P(E)** represents the probability of an event.
- **Frequency** is the number of times a category appears in the dataset.
- **Total Number of Records** is the total number of observations present in the dataset.

The probability values always satisfy the following condition:

$$0 \leq P(E) \leq 1$$

The sum of the probabilities of all categories is always equal to:

$$\sum P(E) = 1$$

In Python, the **Pandas** library provides the `value_counts()` function to calculate the frequency of each category, making the computation of probability distributions simple and efficient.

Software Requirements

- Operating System: Windows 10/11 or Linux
- Python 3.x
- Anaconda Distribution
- Jupyter Notebook
- Pandas Library
- CSV Dataset: **players_info.csv**

Formula

The probability of each category is calculated using:

$$Probability = \frac{Frequency}{Total\ Number\ of\ Records}$$

Percentage is calculated using:

$$Percentage = Probability \times 100$$

Algorithm

Step 1: Start Jupyter Notebook using Anaconda Navigator.

Step 2: Create a new Python Notebook.

Step 3: Import the Pandas library.

Step 4: Read the dataset `players_info.csv` using the `read_csv()` function.

Step 5: Display the first five records of the dataset.

Step 6: Display the column names of the dataset.

Step 7: Find the total number of records in the dataset.

Step 8: Calculate the frequency of each category using the `value_counts()` function.

Step 9: Calculate the probability by dividing the frequency by the total number of records.

Step 10: Convert the probability into percentage.

Step 11: Display the results in tabular format.

Step 12: Verify that the sum of all probabilities is equal to 1.

Procedure

Step 1

Open **Anaconda Navigator**.

Step 2

Launch **Jupyter Notebook**.

Step 3

Create a new **Python 3 Notebook**.

Step 4

Save the notebook with the name:

Experiment_01_Probability_Distribution.ipynb

Step 5

Copy the dataset **players_info.csv** into the same folder where the notebook is saved.

Step 6

Import the Pandas library.

import pandas as pd

Step 7

Read the CSV dataset.

df = pd.read_csv("players_info.csv")

Step 8

Display the first five records.

print(df.head())

This step verifies that the dataset has been loaded successfully.

Step 9

Display the column names.

print(df.columns)

This helps identify the available columns in the dataset.

Step 10

Find the total number of records.

total_players = len(df)

This value is used as the denominator while calculating probabilities.

Step 11

Calculate the frequency of each category using the `value_counts()` function.

Example:

gender_frequency = df["gender"].value_counts()

The `value_counts()` function counts the number of occurrences of each category.

Step 12

Calculate the probability of each category.

Example:

gender_probability = gender_frequency / total_players

The probability is calculated by dividing the frequency by the total number of records.

Step 13

Convert the probability into percentage.

percentage = gender_probability * 100

Step 14

Display the frequency, probability, and percentage in a tabular format.

Step 15

Repeat Steps 11–14 for the remaining categorical variables:

- Batting Style
- Bowling Style
- Player Position
- Country

Step 16

Verify the probability distribution.

print(gender_probability.sum())

The output should be: 1.0

Similarly, verify the probability distribution for all the remaining categories.

Python Program

```
# Experiment 1
# Program to calculate the probability distribution
# of Gender, Batting Style, Bowling Style,
# Position and Country.

# Import the Pandas library
import pandas as pd

# Read the dataset
df = pd.read_csv("players_info.csv")
```

```
# Display the first five records
print("First Five Records")
print(df.head())

# Display the column names
print("\nColumn Names")
print(df.columns)

# Find the total number of players
total_players = len(df)

print("\nTotal Number of Players =", total_players)
# ---------------------------------------
# a) Probability Distribution of Gender
# ---------------------------------------
print("\na) Probability Distribution of Gender")
gender_frequency = df["gender"].value_counts()
gender_probability = gender_frequency / total_players
gender_result = pd.DataFrame({
    "Frequency": gender_frequency,
    "Probability": gender_probability,
    "Percentage": gender_probability * 100
})

print(gender_result)
print("\nTotal Probability =", gender_probability.sum())
```

```
# ---------------------------------------
# b) Probability Distribution of Batting Style
# ---------------------------------------
print("\nb) Probability Distribution of Batting Style")
batting_frequency = df["batting_style"].value_counts()
batting_probability = batting_frequency / total_players

batting_result = pd.DataFrame({
    "Frequency": batting_frequency,
    "Probability": batting_probability,
    "Percentage": batting_probability * 100
})
print(batting_result)
print("\nTotal Probability =", batting_probability.sum())
# ---------------------------------------
# c) Probability Distribution of Bowling Style
# ---------------------------------------
print("\nc) Probability Distribution of Bowling Style")
bowling_frequency = df["bowling_style"].value_counts()
bowling_probability = bowling_frequency / total_players

bowling_result = pd.DataFrame({
    "Frequency": bowling_frequency,
    "Probability": bowling_probability,
    "Percentage": bowling_probability * 100
})
```

```
print(bowling_result)
print("\nTotal Probability =", bowling_probability.sum())
# ---------------------------------------
# d) Probability Distribution of Position
# ---------------------------------------
print("\nd) Probability Distribution of Player Position")
position_frequency = df["position"].value_counts()
position_probability = position_frequency / total_players

position_result = pd.DataFrame({
    "Frequency": position_frequency,
    "Probability": position_probability,
    "Percentage": position_probability * 100
})
print(position_result)
print("\nTotal Probability =", position_probability.sum())
# ---------------------------------------
# e) Probability Distribution of Country
# ---------------------------------------
print("\ne) Probability Distribution of Country")
country_frequency = df["country"].value_counts()
country_probability = country_frequency / total_players
country_result = pd.DataFrame({
    "Frequency": country_frequency,
    "Probability": country_probability,
    "Percentage": country_probability * 100
})
```

```
print(country_result)

print("\nTotal Probability =",

    country_probability.sum())
```

Expected Output

The program should display:

- First five records of the dataset.
- Column names.
- Total number of records.
- Frequency table for each categorical variable.
- Probability table for each categorical variable.
- Percentage distribution.
- Sum of probabilities equal to 1.0.

Result

The probability distributions of Gender, Batting Style, Bowling Style, Player Position, and Country were successfully calculated from the `players_info.csv` dataset using Python and the Pandas library. The calculated probabilities were verified, and the sum of probabilities for each categorical variable was found to be equal to 1, confirming the correctness of the probability distributions.

Viva Questions

1. What is probability?
2. What is a probability distribution?
3. What is the formula for calculating probability?
4. Why is Pandas used in Python?
5. What is a DataFrame?
6. What is the use of value_counts()?
7. Why is read_csv() used?
8. What does len(df) return?
9. Why should the sum of all probabilities be equal to 1?
10. Explain the output of your program.

OUTPUT:

```
First Five Records
  player_id   player_name gender batting_style   bowling_style      position  \
0      P001  Aarav Sharma   Male        Switch  Right-arm pace        Batter
1      P002   Rohan Singh   Male    Right-hand  Right-arm pace        Bowler
2      P003  Grace Sharma   Male     Left-hand  Right-arm spin        Batter
3      P004  Arjun Sharma   Male     Left-hand  Right-arm spin  Wicketkeeper
4      P005   Kabir Brown   Male    Right-hand  Right-arm pace        Batter

     country
0   Pakistan
1      India
2  Australia
3  Australia
4  Australia

Column Names
Index(['player_id', 'player_name', 'gender', 'batting_style', 'bowling_style',
       'position', 'country'],
      dtype='object')

Total Number of Players = 80

a) Probability Distribution of Gender
        Frequency  Probability  Percentage
gender
Male           56          0.7        70.0
Female         24          0.3        30.0

Total Probability = 1.0
```

```
b) Probability Distribution of Batting Style
              Frequency  Probability  Percentage
batting_style
Right-hand            48         0.60        60.0
Left-hand             28         0.35        35.0
Switch                 4         0.05         5.0

Total Probability = 1.0

c) Probability Distribution of Bowling Style
                   Frequency  Probability  Percentage
bowling_style
Right-arm pace            24        0.300        30.0
Right-arm spin            20        0.250        25.0
Left-arm pace             14        0.175        17.5
Left-arm spin             12        0.150        15.0
Wicketkeeper/None         10        0.125        12.5

Total Probability = 1.0

d) Probability Distribution of Player Position
              Frequency  Probability  Percentage
position
Batter               24        0.300        30.0
Bowler               22        0.275        27.5
All-rounder          20        0.250        25.0
Wicketkeeper         14        0.175        17.5

Total Probability = 1.0
```

```
e) Probability Distribution of Country
              Frequency  Probability  Percentage
country
India                20       0.2500       25.00
Australia            12       0.1500       15.00
England              12       0.1500       15.00
South Africa         10       0.1250       12.50
New Zealand           8       0.1000       10.00
West Indies           7       0.0875        8.75
Pakistan              6       0.0750        7.50
Sri Lanka             5       0.0625        6.25

Total Probability = 1.0
```

EXPERIMENT NO. 2

Title

Binomial Probability Distribution Using Python

Aim

To calculate the probability of different outcomes using the **Binomial Probability Distribution** and visualize the probability distribution using Python.

Problem Statement

80% of all the visitors to **Museum of Goa** end up buying souvenirs from the souvenir shop at the museum.

On the coming Sunday, if a random sample of **10 visitors** is selected, determine:

a) The probability that **every visitor** will buy a souvenir.

b) The probability that **at most (maximum) 7 visitors** will buy souvenirs.

Also display the complete probability distribution and plot the Binomial Distribution graph.

Learning Objectives

After completing this experiment, the student will be able to:

- Understand Binomial Probability Distribution.
- Calculate probabilities using the Binomial Formula.
- Apply mathematical concepts using Python.
- Display probability distributions in tabular form.
- Visualize probability distributions using a bar chart.
- Interpret probability values.

Theory

The **Binomial Distribution** is a discrete probability distribution that describes the probability of obtaining exactly **x successes** in **n independent trials**, where each trial has only two possible outcomes:

- Success
- Failure

The probability of success remains constant for every trial.

A Binomial Distribution is applicable when:

- There are a fixed number of trials.
- Each trial has only two outcomes.
- The probability of success remains constant.
- Each trial is independent.

In this experiment,

- Success = Visitor buys a souvenir.
- Failure = Visitor does not buy a souvenir.

Since every visitor either buys or does not buy a souvenir, the problem follows a **Binomial Distribution**.

Software Requirements

- Windows 10/11
- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas Library
- Matplotlib Library

Algorithm

Step 1: Start Jupyter Notebook.

Step 2: Import the required Python libraries.

Step 3: Initialize the values of **n**, **p**, and **q**.

Step 4: Calculate the probability that all visitors buy souvenirs.

Step 5: Calculate probabilities for 0 to 7 visitors.

Step 6: Add these probabilities to obtain the probability of at most 7 visitors buying souvenirs.

Step 7: Calculate the probabilities for all values from 0 to 10.

Step 8: Store the probabilities in a DataFrame.

Step 9: Display the probability distribution.

Step 10: Plot the Binomial Distribution graph.

Procedure

Step 1

Open **Anaconda Navigator**.

Step 2

Launch **Jupyter Notebook**.

Step 3

Create a new Python Notebook.

Step 4

Import the required libraries.

```
from math import comb
import pandas as pd
import matplotlib.pyplot as plt
```

Step 5

Assign the values.

n=10

p=0.80

q=0.20

Step 6

Calculate the probability that every visitor buys a souvenir.

Step 7

Calculate the probability that at most seven visitors buy souvenirs.

Step 8

Prepare the probability distribution table.

Step 9

Display the probability distribution.

Step 10

Plot the Binomial Distribution graph.

Python Program

```
# Experiment 2
# Binomial Probability Distribution

from math import comb
import pandas as pd
import matplotlib.pyplot as plt

# Number of visitors
n = 10

# Probability of buying a souvenir
p = 0.80
```

```
# Probability of not buying
q = 1 - p

# ------------------------------------
# Question 1
# Probability that every visitor buys
# ------------------------------------

x = 10

prob_everyone = comb(n, x) * (p ** x) * (q ** (n - x))

print("QUESTION 1")
print("Probability that every visitor buys a souvenir =", prob_everyone)
print("Percentage =", prob_everyone * 100)

# ------------------------------------
# Question 2
# Probability that at most 7 visitors buy
# ------------------------------------

prob_max_7 = 0

probability_values = []

for x in range(11):

    probability = comb(n, x) * (p ** x) * (q ** (n - x))

    probability_values.append(probability)

    if x <= 7:
        prob_max_7 += probability

print("\nQUESTION 2")
print("Probability that at most 7 visitors buy =", prob_max_7)
print("Percentage =", prob_max_7 * 100)
```

```
# ----------------------------------
# Probability Distribution Table
# ----------------------------------

distribution = pd.DataFrame({ "Visitors Buying": range(11), "Probability":
probability_values })

print("\nComplete Probability Distribution")

print(distribution)

print("\nTotal Probability =", sum(probability_values))

# ----------------------------------
# Final Answers
# ----------------------------------

print("\nFINAL ANSWERS")

print("P(X=10) =", round(prob_everyone,4))

print("P(X<=7) =", round(prob_max_7,4))

# ----------------------------------
# Graph
# ----------------------------------

plt.figure(figsize=(8,5))

plt.bar(range(11), probability_values)

plt.xlabel("Number of Visitors Buying Souvenirs")

plt.ylabel("Probability")

plt.title("Binomial Probability Distribution")

plt.xticks(range(11))

plt.grid(axis="y")

plt.show()
```

Explanation of the Program

`comb(n,x)`

Calculates the binomial coefficient

$$\binom{n}{x}$$

`p**x`

Calculates

$$p^x$$

`q**(n-x)`

Calculates

$$q^{(n-x)}$$

`probability_values.append(probability)`

Stores every probability from **0 to 10**.

`pd.DataFrame()`

Displays the probability distribution in table format.

`plt.bar()`

Draws the Binomial Distribution graph.

Expected Output

```
QUESTION 1

Probability that every visitor buys a souvenir = 0.1073741824

Percentage = 10.73741824
```

```
QUESTION 2

Probability that at most 7 visitors buy = 0.3222004736

Percentage = 32.22004736
```

```
FINAL ANSWERS

P(X=10)=0.1074

P(X<=7)=0.3222
```

The program will also display:

- Complete probability distribution table.
- Binomial Distribution bar chart.

Interpretation

- The probability that **all 10 visitors** buy souvenirs is approximately **0.1074 (10.74%)**.
- The probability that **at most 7 visitors** buy souvenirs is approximately **0.3222 (32.22%)**.
- The probability distribution table shows the probability for every possible number of visitors (0–10) buying souvenirs.
- The bar chart visually represents the Binomial Distribution, with the highest probability occurring around the expected number of buyers.

Result

The Binomial Probability Distribution was successfully used to calculate the probability that every visitor buys a souvenir and the probability that at most seven visitors buy souvenirs. The probability distribution was displayed in tabular form and visualized using a bar chart.

OUTPUT:

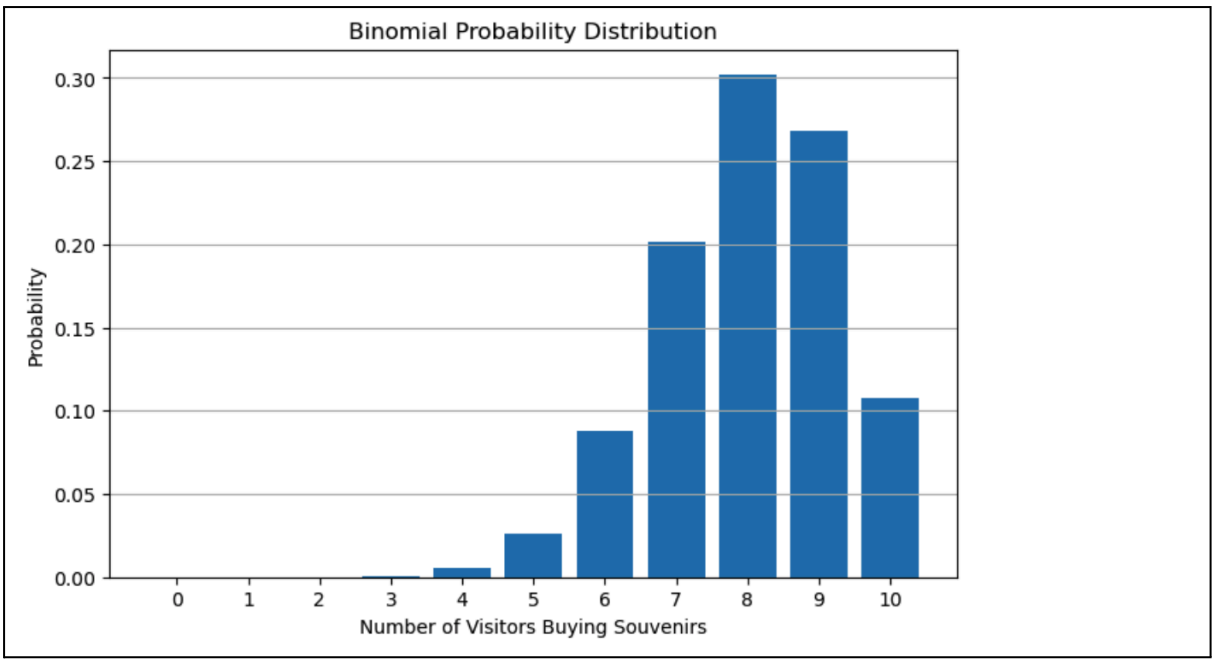
```
QUESTION 1
Probability that every visitor buys a souvenir = 0.10737418240000006
Percentage = 10.737418240000006

QUESTION 2
Probability that at most 7 visitors buy = 0.32220047359999987
Percentage = 32.22004735999999

Complete Probability Distribution
    Visitors Buying   Probability
0                 0  1.024000e-07
1                 1  4.096000e-06
2                 2  7.372800e-05
3                 3  7.864320e-04
4                 4  5.505024e-03
5                 5  2.642412e-02
6                 6  8.808038e-02
7                 7  2.013266e-01
8                 8  3.019899e-01
9                 9  2.684355e-01
10               10  1.073742e-01

Total Probability = 1.0

FINAL ANSWERS
P(X=10) = 0.1074
P(X<=7) = 0.3222
```



Viva Questions

1. What is a Binomial Distribution?
2. What are the conditions for a Binomial experiment?
3. What is meant by a Bernoulli trial?
4. What is the difference between success and failure?
5. What is the value of **q** if **p = 0.8**?
6. What does `comb()` calculate?
7. Why is `range(11)` used?
8. Why should the total probability equal 1?
9. Why is a bar graph suitable for this distribution?
10. Give two real-life applications of the Binomial Distribution.

EXPERIMENT NO. 3

Title

Probability Distribution of SAT Scores Using Python

Aim

To calculate the probability that a student scores **less than 800** and **more than 1300** in the SAT examination using Python.

Problem Statement

A testing agency wants to analyze the complexity of the **SAT Exam 2022**. They have collected the SAT scores of **1000 students** in the file **sat_score.csv**.

Determine:

a) The probability that a student scores **less than 800**.

b) The probability that a student scores **more than 1300**.

Learning Objectives

After completing this experiment, the student will be able to:

- Read a CSV dataset using Python.
- Calculate probabilities from real-world data.
- Understand frequency and probability.
- Display probability values.
- Visualize the data using a histogram.

Theory

Probability is the measure of the likelihood of an event occurring.

For a dataset,

$$P(E) = \frac{\text{Number of favourable observations}}{\text{Total number of observations}}$$

where

- **Favourable observations** = Number of students satisfying the condition.
- **Total observations** = Total number of students.

In this experiment,

- Event 1 = Student scores less than 800.
- Event 2 = Student scores more than 1300.

The probability is obtained by counting the number of students satisfying the condition and dividing it by the total number of students.

Formula

Probability Formula

$$P(E) = \frac{\text{Frequency of Event}}{\text{Total Number of Records}}$$

For Question (a)

$$P(SAT < 800) = \frac{\text{Students scoring below 800}}{1000}$$

For Question (b)

$$P(SAT > 1300) = \frac{\text{Students scoring above 1300}}{1000}$$

Software Requirements

- Windows 10/11
- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas
- Matplotlib
- sat_score.csv

Algorithm

Step 1: Open Jupyter Notebook.

Step 2: Import the required libraries.

Step 3: Read the dataset.

Step 4: Display the first five records.

Step 5: Find the total number of students.

Step 6: Count the number of students scoring below 800.

Step 7: Count the number of students scoring above 1300.

Step 8: Calculate the probabilities.

Step 9: Display the probabilities.

Step 10: Plot the distribution of SAT scores.

Procedure

Step 1

Open **Anaconda Navigator**.

Step 2

Launch **Jupyter Notebook**.

Step 3

Create a new Python Notebook.

Step 4

Import the required libraries.

```
import pandas as pd

import matplotlib.pyplot as plt
```

Step 5

Read the dataset.

```
df = pd.read_csv("sat_score.csv")
```

Step 6

Display the first five records.

print(df.head())

Step 7

Display the column names.

print(df.columns)

Step 8

Find the total number of students.

total_students = len(df)

Step 9

Count the students scoring below 800.

less_800 = len(df[df["SAT_score"] < 800])

Step 10

Count the students scoring above 1300.

more_1300 = len(df[df["SAT_score"] > 1300])

Step 11

Calculate the probabilities.

prob_less_800 = less_800 / total_students

prob_more_1300 = more_1300 / total_students

Step 12

Display the results.

Step 13

Draw the histogram.

Python Program

Probability of SAT Scores

```
import pandas as pd
import matplotlib.pyplot as plt

# Read dataset
df = pd.read_csv("sat_score.csv")

# Display first five records
print("First Five Records")
print(df.head())
# Display column names
print("\nColumn Names")
print(df.columns)

# Total number of students
total_students = len(df)
print("\nTotal Students =", total_students)

# Students scoring below 800
less_800 = len(df[df["SAT_score"] < 800])

# Students scoring above 1300
more_1300 = len(df[df["SAT_score"] > 1300])

# Calculate probabilities
prob_less_800 = less_800 / total_students
prob_more_1300 = more_1300 / total_students
```

Display results

print("\nStudents scoring below 800 =", less_800)

print("Probability =", prob_less_800)

print("Percentage =", prob_less_800 * 100)

print("\nStudents scoring above 1300 =", more_1300)

print("Probability =", prob_more_1300)

print("Percentage =", prob_more_1300 * 100)

Histogram

plt.figure(figsize=(8,5))

plt.hist(df["SAT_score"], bins=20)

plt.title("Distribution of SAT Scores")

plt.xlabel("SAT Score")

plt.ylabel("Number of Students")

plt.grid(True)

plt.show()

Explanation of the Program

- `pd.read_csv()` reads the dataset.
- `len(df)` calculates the total number of students.
- `df["SAT_score"] < 800` filters students scoring below 800.
- `df["SAT_score"] > 1300` filters students scoring above 1300.
- Probability is calculated by dividing the number of favourable students by the total number of students.
- `plt.hist()` creates a histogram to visualize the score distribution.

OUTPUT:

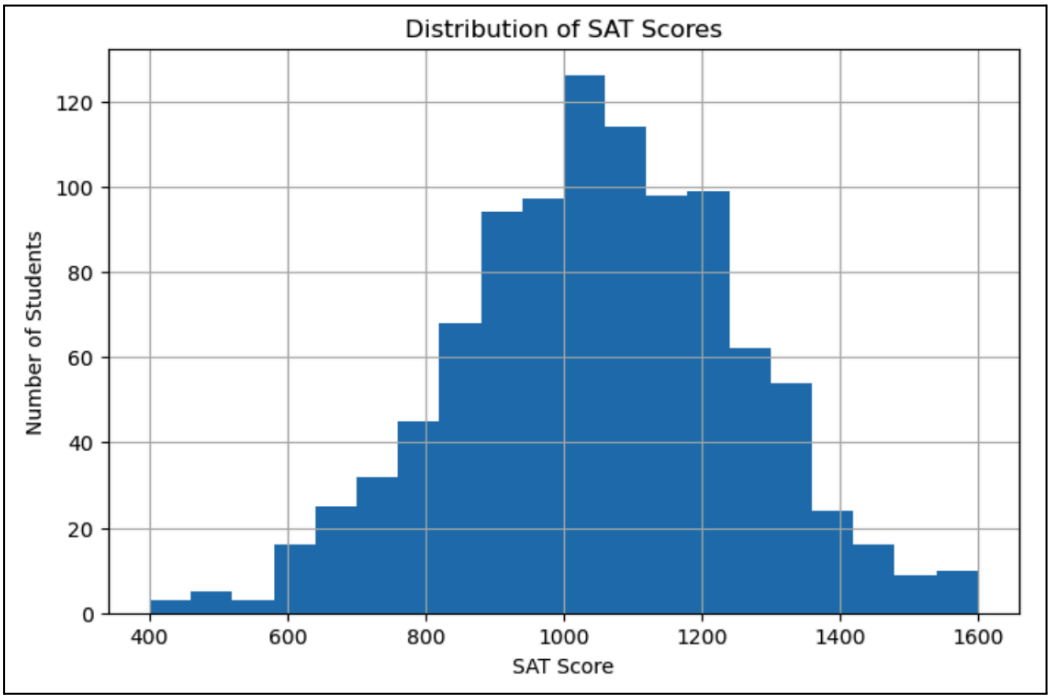
```
First Five Records
  student_id  SAT_score
0      S0001       1037
1      S0002        920
2      S0003        682
3      S0004       1188
4      S0005       1129

Column Names
Index(['student_id', 'SAT_score'], dtype='object')

Total Students = 1000

Students scoring below 800 = 109
Probability = 0.109
Percentage = 10.9

Students scoring above 1300 = 112
Probability = 0.112
Percentage = 11.200000000000001
```



Interpretation

- Out of **1000 students**, **109 students** scored **less than 800**.
- Therefore,

$$P(SAT < 800) = 0.109$$

- Out of **1000 students**, **112 students** scored **more than 1300**.
- Therefore,

$$P(SAT > 1300) = 0.112$$

These probabilities indicate that approximately **10.9%** of the students scored below 800, while **11.2%** scored above 1300.

Result

The probabilities of students scoring **less than 800** and **more than 1300** in the SAT examination were successfully calculated from the given dataset using Python. The results show that the probability of scoring below 800 is **0.109 (10.9%)**, and the probability of scoring above 1300 is **0.112 (11.2%)**. The distribution of SAT scores was also visualized using a histogram.

Viva Questions

1. What is probability?
2. What is a probability distribution?
3. What is a CSV file?
4. What is the purpose of `read_csv()`?
5. What does `len(df)` return?
6. How do you filter records based on a condition in Pandas?
7. How is the probability of an event calculated from a dataset?
8. What does `plt.hist()` do?
9. Why is a histogram used for SAT scores?
10. What is the probability of a student scoring less than 800 in this dataset?
11. What is the probability of a student scoring more than 1300 in this dataset?
12. What conclusion can you draw from the calculated probabilities?

EXPERIMENT NO. 4

Title

One-Sample t-Test for Mean Mobile Internet Usage Time

Aim

To determine whether there is significant evidence that the population mean time spent per day accessing the Internet through a mobile device is **different from 144 minutes**, using a **One-Sample t-Test** at a **5% level of significance**.

Problem Statement

A Marketing services company reported that the typical American spends a mean of **144 minutes (2.4 hours) per day** accessing the Internet via a mobile device.

A sample of **30 friends and family members** was selected, and their mobile Internet access times are stored in the CSV file `MobileTime.csv`.

Determine whether there is evidence that the **population mean time** spent per day accessing the Internet via a mobile device is **different from 144 minutes**.

Use: $\alpha=0.05$

Learning Objectives

After completing this experiment, the student will be able to:

- Understand the concept of hypothesis testing.
- Understand the One-Sample t-Test.
- Formulate null and alternative hypotheses.
- Calculate a sample mean and sample standard deviation.
- Calculate the t-statistic.
- Interpret the p-value.
- Make a statistical decision based on the level of significance.
- Draw a conclusion from the hypothesis test using Python.

Theory

A **hypothesis test** is a statistical method used to determine whether there is sufficient evidence in a sample to support a claim about a population.

In this experiment, the company claims that the mean mobile Internet usage time is: μ=144

We have a sample of 30 observations and want to determine whether the actual population mean is **different from 144 minutes**.

Since:

- There is only **one sample**.
- The population mean to be tested is **144 minutes**.
- The population standard deviation is not given.
- The sample size is 30.

We use a **One-Sample t-Test**.

Hypotheses

Since the question asks whether the population mean is **different from 144 minutes**, this is a **two-tailed test**.

Null Hypothesis

$$H_0 : \mu = 144$$

The population mean mobile Internet usage time is 144 minutes.

Alternative Hypothesis

$$H_1 : \mu \neq 144$$

The population mean mobile Internet usage time is different from 144 minutes.

Level of Significance

The given level of significance is:

$$\alpha = 0.05$$

Therefore, the test will be conducted at the **5% significance level**.

Formula

The One-Sample t-Test statistic is calculated using:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

where:

- $\bar{x}$ = Sample mean
- μ_0 = Hypothesized population mean
- s = Sample standard deviation
- n = Sample size

In this experiment:

$$\mu_0 = 144$$

and

$$n = 30$$

Decision Rule

The decision is based on the **p-value**.

If:

$$p\text{-value} < \alpha$$

then **Reject** H_0.

If:

$$p\text{-value} \geq \alpha$$

then **Fail to Reject** H_0.

Here:

$$\alpha = 0.05$$

Software Requirements

- Python 3.x
- Anaconda Distribution
- Jupyter Notebook
- Pandas Library
- SciPy Library
- Matplotlib Library
- `MobileTime.csv`

Dataset Description

The supplied dataset contains **30 observations**.

Column	Description
`participant_id`	Unique identification number of each participant
`MobileTime`	Time spent accessing the Internet through a mobile device, in minutes

The dataset contains:

$$n = 30$$

observations.

Algorithm

Step 1: Start Jupyter Notebook.

Step 2: Import the required Python libraries.

Step 3: Read the InternetMobileTime.csv dataset.

Step 4: Display the first five records.

Step 5: Display the column names.

Step 6: Find the total number of observations.

Step 7: Calculate the sample mean of MobileTime.

Step 8: Perform a One-Sample t-Test by comparing the sample mean with 144 minutes.

Step 9: Obtain the t-statistic and p-value.

Step 10: Compare the p-value with the significance level $\alpha=0.05$.

Step 11: Reject or fail to reject the null hypothesis.

Step 12: Interpret the result and state the conclusion.

Procedure

Step 1: Open Anaconda Navigator

Open **Anaconda Navigator** from your computer.

Step 2: Launch Jupyter Notebook

Open **Jupyter Notebook**.

Step 3: Create a New Notebook

Create a new Python notebook.

Save the notebook as: Experiment_04_One_Sample_t_Test.ipynb

Step 4: Place the Dataset

Place the file: MobileTime.csv

in the same folder as the Jupyter Notebook.

Step 5: Import the Required Libraries

Enter: import pandas as pd

from scipy.stats import ttest_1samp

Step 6: Read the Dataset

df = pd.read_csv("InternetMobileTime.csv")

Step 7: Display the Dataset

print(df.head())

This displays the first five observations.

Step 8: Check the Column Names

print(df.columns)

The dataset should display:

Index(['participant_id', 'MobileTime'], dtype='object')

Step 9: Find the Number of Observations

n = len(df)

The result is: 30

Step 10: Calculate the Sample Mean

sample_mean = df["MobileTime"].mean()

For the supplied dataset: $\bar{x}$=156.1333 minutes.

Step 11: Perform the One-Sample t-Test

The hypothesized mean is 144 minutes.

t_statistic, p_value = ttest_1samp(df["MobileTime"], 144)

Step 12: Compare the p-value with 0.05

If the p-value is less than 0.05, reject the null hypothesis.

Python Program

```
# Experiment 4
# One-Sample t-Test

import pandas as pd
from scipy.stats import ttest_1samp

# Read the dataset
df = pd.read_csv("InternetMobileTime.csv")

# Display first five records
print("First Five Records")
print(df.head())

# Display column names
print("\nColumn Names")
print(df.columns)

# Find total number of observations
n = len(df)

print("\nTotal Number of Observations =", n)

# Calculate sample mean
sample_mean = df["MobileTime"].mean()

print("\nSample Mean =", sample_mean)

# Hypothesized population mean
population_mean = 144

# Perform One-Sample t-Test
t_statistic, p_value = ttest_1samp(
    df["MobileTime"],
    population_mean
)
```

```
# Display test results
print("\nOne-Sample t-Test Results")

print("t-statistic =", t_statistic)

print("p-value =", p_value)

# Level of significance
alpha = 0.05

print("\nLevel of Significance =", alpha)

# Decision
if p_value < alpha:

    print("\nReject the Null Hypothesis")

    print("Conclusion: There is sufficient evidence that")
    print("the population mean mobile Internet usage time")
    print("is different from 144 minutes.")

else:

    print("\nFail to Reject the Null Hypothesis")

    print("Conclusion: There is not sufficient evidence that")
    print("the population mean mobile Internet usage time")
    print("is different from 144 minutes.")
```

Expected Output

For your supplied dataset, the important output will be approximately:

First Five Records

	participant_id	MobileTime
0	M01	132
1	M02	148
2	M03	151
3	M04	167
4	M05	143

Total Number of Observations = 30

Sample Mean = 156.13333333333333

One-Sample t-Test Results

t-statistic = 5.812140901078351

p-value = 2.6699618932126e-06

Level of Significance = 0.05

Reject the Null Hypothesis

Conclusion: There is sufficient evidence that the population mean mobile Internet usage time is different from 144 minutes.

Explanation of the Python Program

1. Import Pandas

import pandas as pd

Pandas is used to read and work with the CSV dataset.

2. Import the One-Sample t-Test

from scipy.stats import ttest_1samp

`ttest_1samp()` is the SciPy function used to perform a One-Sample t-Test.

3. Read the CSV File

df = pd.read_csv("InternetMobileTime.csv")

This reads the dataset and stores it in the DataFrame `df`.

4. Find the Number of Observations

n = len(df)

`len(df)` returns the number of rows in the dataset. For this dataset: n=30

5. Calculate the Sample Mean

sample_mean = df["MobileTime"].mean()

The `.mean()` function calculates the average mobile Internet usage time.

For the given dataset: $\bar{x}=156.1333$ minutes.

6. Specify the Hypothesized Mean

population_mean = 144

The value 144 comes from the problem statement.

7. Perform the t-Test

t_statistic, p_value = ttest_1samp(df["MobileTime"], population_mean)

This compares the sample mean with the hypothesized population mean of 144 minutes.

The function returns two values:

- **t-statistic**
- **p-value**

8. Set the Significance Level

alpha = 0.05

The problem specifies a significance level of 5%.

9. Make the Decision

if p_value < alpha:

If the p-value is less than 0.05, the null hypothesis is rejected.

Interpretation of the Output

The calculated values are:

$$t = 5.8121$$

and

$$p = 2.67 \times 10^{-6}$$

or approximately:

$$p = 0.00000267$$

Compare the p-value with the significance level:

$$0.00000267 < 0.05$$

Therefore, we **reject the null hypothesis**.

The sample mean is:

$$156.13 \text{ minutes}$$

which is higher than the reported mean of 144 minutes.

Statistical Conclusion

Since the p-value is less than 0.05, there is **statistically significant evidence** that the population mean time spent accessing the Internet through a mobile device is **different from 144 minutes per day**.

Therefore:

$$\boxed{\text{Reject } H_0}$$

and

$$\boxed{\text{Accept } H_1}$$

in the practical sense that the data provide evidence for the alternative hypothesis.

Result

The **One-Sample t-Test** was successfully performed on the `MobileTime.csv` dataset.

At the **5% level of significance**, the calculated p-value is approximately **0.00000267**, which is less than 0.05. Hence, the null hypothesis is rejected.

Therefore, there is **significant evidence that the population mean time spent accessing the Internet via a mobile device is different from 144 minutes per day**.

Viva Questions – Experiment 4

Basic Questions

1. What is hypothesis testing?
2. What is a One-Sample t-Test?
3. Why is a One-Sample t-Test used in this experiment?
4. What is the population mean given in the problem?
5. What is the sample size?
6. What is the sample mean obtained from the dataset?
7. What is the level of significance?
8. What is the null hypothesis?
9. What is the alternative hypothesis?
10. Is this a one-tailed or two-tailed test? Why?

Formula and Statistics

11. What is the formula for the One-Sample t-Test?
12. What is a t-statistic?
13. What is a p-value?
14. What are the degrees of freedom in this experiment?
15. How are degrees of freedom calculated?
16. What does the significance level α=0.05 mean?

Python Questions

17. Which library is used to perform the t-Test?
18. What is the purpose of `ttest_1samp()`?
19. What does the following statement do? df["MobileTime"].mean()
20. What does `len(df)` return?
21. Why is `144` passed to `ttest_1samp()`?
22. What does the `p_value` represent?
23. What happens if the p-value is less than 0.05?
24. What happens if the p-value is greater than 0.05?

Dataset/Result Questions

25. What is the sample mean in this experiment?
26. What is the calculated t-statistic?
27. What is the calculated p-value?
28. Is the p-value less than 0.05?
29. Should the null hypothesis be rejected?
30. What is your final conclusion from the experiment?

Most Important Viva Question

Why is this called a two-tailed test?

Answer: The question asks whether the population mean is **"different from" 144 minutes**. It does not specifically ask whether it is greater than or less than 144. Therefore, both directions are considered: H1:μ=144 Hence, it is a **two-tailed One-Sample t-Test**.